Facultad de Ingeniería

Ingeniería en Informática y Sistemas

Área de Informática

Manejo e Implementación de Archivo

Proyecto 2

Rodrigo Andrés Pinto Ticú

1507325

Quetzaltenango, Quetzaltenango 4 de noviembre del 2026

Estructura del registro

El registro se almacena con una longitud fija de 100 bytes, la distribución es la siguiente:

- ID = 4 bytes (int)
- NAME = 30 bytes (string)
- PHONE = 9 bytes (string)
- EMAIL = 17 bytes (string)
- CITY = 40 bytes (string)

```
__FORMAT = "<i30s9s17s40s"
```

Tamaño total del registro: 100 bytes

Implementación Seek() y Tell():

Al ser registros de longitud fija, en el sistema se utiliza del index.json para saber la posición en la que debe de apuntar el puntero del archivo para obtener los próximos 100 bites y leer el registro completo

```
def read(self, format:str, size:int, pos):
    with open(self.PATH, 'rb') as file:
        file.seek(pos)
        b = file.read(size)
        data = struct.unpack(format, b)

    return Contact(
        data[0],
        data[1].decode('utf-8').rstrip('\x00'),
        data[2].decode('utf-8').rstrip('\x00'),
        data[3].decode('utf-8').rstrip('\x00'),
        data[4].decode('utf-8').rstrip('\x00')
    )
```

Cada vez que se quiere leer un registro, se obtiene la posición del index.json y luego con ayuda de seek() posiciona el puntero en el inicio, luego utilizo read para leer los próximos 100 bytes y con struct.unpack() obtengo nuevamente los datos (en caso de los strings, decodifico a utf-8 y elimino los espacios varios '\x00').

```
def getLastPos(self):
    try:
        with open(self.PATH, 'rb') as file:
            file.seek(0, 2)
            pos = file.tell()
        return pos
    except:
        return 0
```

```python
def setContact(self, contact:Contact, pos):
    self.loadConcatIndex()
    if str(contact.id) in self.__index:
        raise Exception(f"El ID {contact.id} ya se encuentra ocupado")

    self.__index[str(contact.id)] = pos
    try:
        with open(self.PATH_TEMP, 'w', encoding='utf-8') as file:
            json.dump(self.__index, file, indent=4)
```

Para almacenar las posiciones en index.json utilizo tell() para obtener la última posición disponible del archivo posicionando el puntero hasta el final de este, de esta manera, se le asigna esta posición al index.json.

Índice y Index.json:

Como fue mencionado antes, el index.json sirve para ubicar la posición de cada registro de manera instantánea

```
data > {} index.json > ...
1   {
2       "0": 0,
3       "1": 100,
4       "2": 200
5   }
```

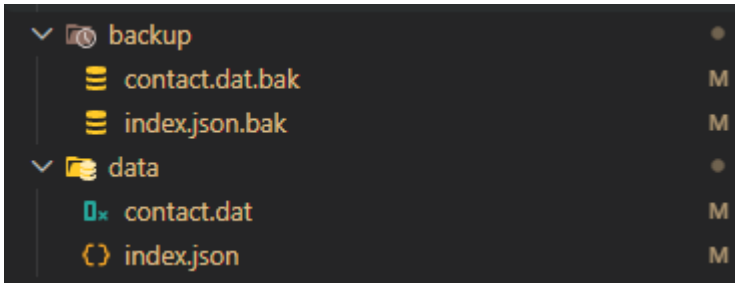
Index.json almacena como clave la ID única de cada registro y como valor la posición entera obtenida con tell(), de esta manera es como se puede realizar cualquier operación como Update o Read con complejidad algorítmica O(1), debido a que se está accediendo al registro directamente y no secuencialmente que tendría una complejidad algorítmica de O(n) en el peor y caso promedio, por lo mismo, el sistema de índices y registro de longitud fija ayuda a que el acceso a la información sea eficiente y no dependa de la cantidad de registros.

Integridad:

```
contacts.sha256
data\contact.dat|702b0c33858654e5b4934700f8a804c70c33ea847360b3a8a16d5af7d6ac6edf
data\index.json|088bca66fdda0d65ec28fea727b49a7e280cfddb06ba0ed9b3ab3213929e2b03
```

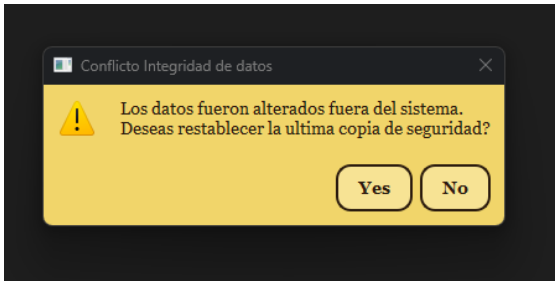
```python
def comparePersistent(self, path) -> int:
    if os.path.exists(str(Path("data") / path)):
        return  0 if self.__hash.compare(path) else 1
    else: return -1
```

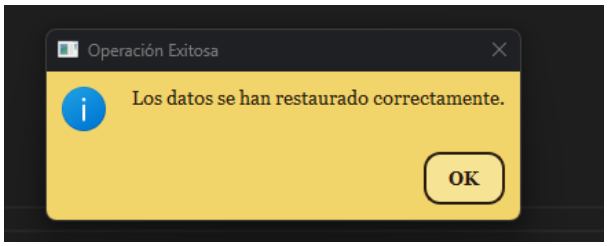
El sistema utiliza sha-256 para obtener el hash de los archivos de guardado, al arranque del sistema, este indica si el archivo fue modificado fuera de la ejecución del programa, sea este el caso, este permite restaurar a una copia de seguridad guardada en la carpeta backup, si se realiza este proceso se recalcula nuevamente el hash para garantizar la integridad.




```
Current:  4ccf915458489955545e7fca96a2d85c52930e4e77897a6593977eea269a3f92
Save:  702b0c33858654e5b4934700f8a804c70c33ea847360b3a8a16d5af7d6ac6edf
Current:  088bca66fdda0d65ec28fea727b49a7e280cfddb06ba0ed9b3ab3213929e2b03
Save:  088bca66fdda0d65ec28fea727b49a7e280cfddb06ba0ed9b3ab3213929e2b03
```

El primero corresponde al contact.dat y el segundo (el que sí es igual porque no se modificó) es el de index.json; pero, al modificarse contact.dat:

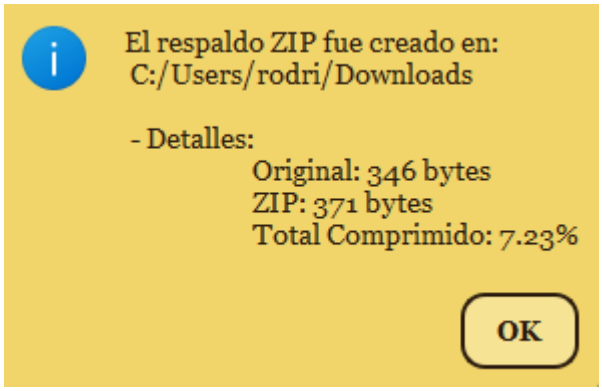





```
Current:  702b0c33858654e5b4934700f8a804c70c33ea847360b3a8a16d5af7d6ac6edf
Save:  702b0c33858654e5b4934700f8a804c70c33ea847360b3a8a16d5af7d6ac6edf
Current:  088bca66fdda0d65ec28fea727b49a7e280cfddb06ba0ed9b3ab3213929e2b03
Save:  088bca66fdda0d65ec28fea727b49a7e280cfddb06ba0ed9b3ab3213929e2b03
```

Vuelve a concordar el hash.

Comparación Zip:



```
origin = sum(f.stat().st_size for f in repo_path.glob('*') if f.is_file())
c = path.stat().st_size

r = (abs(origin - c) / origin) * 100 if origin > 0 else 0

data = {
    "origin": origin,
    "zip": c,
    "per": max(0, r)
}
```

Al crear el .zip se calcula el tamaño original de la carpeta data en general y el del zip, luego a manera de porcentaje se obtiene el cociente de la diferencia entre lo original y el tamaño del zip con el tamaño original, quedando:

$$\frac{|346 - 371|}{346} * 100 = 7.23\%$$

Que al ser una carpeta bastante liviana ahora que no hay demasiados registros, termina siendo un porcentaje de tamaño reducido poco considerable, pero que puede aumentar si es que aumenta la cantidad de registros.

Capturas de pantalla:

MainMenu



Agregar contacto



Buscar Contacto

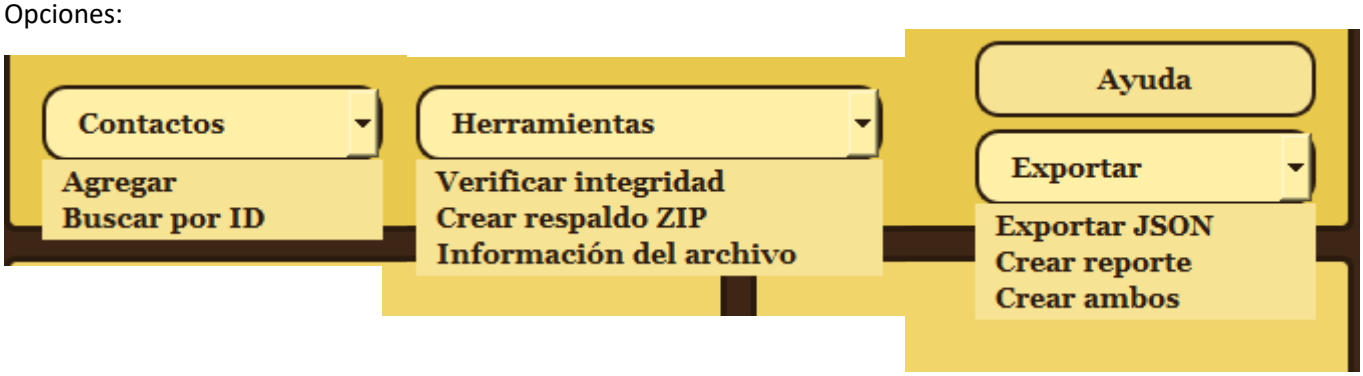


Edición



Comparte menú con el de búsqueda

Opciones:



Información del archivo (metadatos):



Ayuda



Reporte generado

REPORTE DE CONTACTOS

ID	NOMBRE	TELÉFONO	CORREO	CIUDAD
0	NUEVO NOMBREEEEEEEEEEEEEEEEEEEE		111111a@aaaaaa.aa	aaaaaaa
1	hola	3423-4234	nuevoc@gmail.com	ciudaddadadaa
2	pepnillo	9599-6666	esto@esuncor.reo	Solola
3	Rodrigo Pinto	3222-2333	correo@url.edu.gt	Quetzaltenango

Json exportado

```
[
    {
        "id": 0,
        "name": "NUEVO NOMBREEEEEEEEEEEEEEEEEE",
        "phone": "1111-1111",
        "email": "111111a@aaaaaa.aa",
        "city": "aaaaaaa"
    },
    {
        "id": 1,
        "name": "hola",
        "phone": "3423-4234",
        "email": "nuevoc@gmail.com",
        "city": "ciudaddadadaa"
    },
    {
        "id": 2,
        "name": "pepnillo",
        "phone": "9599-6666",
        "email": "esto@esuncor.reo",
        "city": "Solola"
    },
    {
        "id": 3,
        "name": "Rodrigo Pinto",
        "phone": "3222-2333",
        "email": "correo@url.edu.gt",
        "city": "Quetzaltenango"
    }
]
```